

Aplicaciones cliente-servidor



Caso práctico

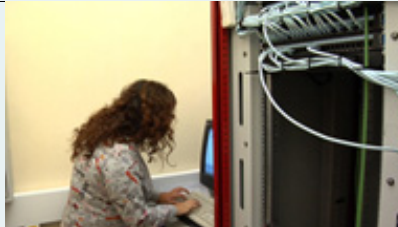
María esta terminado de realizar una aplicación.

María: Hola **Juan**. Ya he visto que estás terminado la aplicación para la empresa. He visto en el código que tan sólo se puede conectar un cliente de forma simultánea. Es necesario

permitir concurrencia para que muchos clientes puedan acceder de forma simultánea a la aplicación. Además, tenemos que asegurar su funcionamiento.

Juan: ¡¡¡¡Qué interesante!!!! Me está gustando mucho la programación de sockets. Pero necesito que me ayudes.

María: No te preocupes. Pero antes de empezar con la parte de programación vamos a ver los conceptos más importantes de las aplicaciones cliente/servidor para que veas todas sus posibilidades.



🔍 [Ministerio de Educación.](#) (Uso educativo [nc](#))

El objetivo de la unidad es que el alumnado profundice en la programación de aplicaciones Cliente/Servidor.

Para ello, primero ampliaremos los conceptos teóricos relacionados con el modelo Cliente/Servidor para más tarde poder aplicar dichos conceptos a la programación en Java de aplicaciones que utilicen Sockets.



Q Ministerio de Educación, Formación Profesional y Deportes

Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes.

 [Aviso Legal](#) 

1.- Paradigma Cliente/Servidor



Caso práctico



Q Ministerio de Educación. (Uso educativo nc)

aplicaciones.

Juan: De acuerdo, estupendo. Estoy deseando empezar...

María: Mira **Juan**. Ahora vamos a ver todas las características del modelo cliente/servidor.

Pero no todo son ventajas. También hay desventajas que tenemos que tener en cuenta a la hora de programar las

En el mundo de las comunicaciones entre equipos el modelo de comunicación más utilizado es el modelo Cliente/servidor ya que ofrece una gran flexibilidad, interoperabilidad y estabilidad para acceder a recursos de forma centralizada.



Q Ministerio de Educación. (Uso educativo nc)

El término modelo Cliente/Servidor

se acuñó por primera vez en los años 80 para explicar un sencillo paradigma: un equipo cliente requiere un servicio de un equipo servidor.

Desde el punto de vista funcional, se puede definir el modelo Cliente/Servidor como una arquitectura distribuida que permite a los usuarios finales obtener acceso a recursos de forma transparente en entornos multiplataforma. Normalmente, los recursos que suele ofrecer el servidor son datos, pero también puede permitir acceso a dispositivos hardware, tiempo de procesamiento, etc.

Los elementos que componen el modelo son:

- **Cliente.** Es el proceso que permite interactuar con el usuario, realizar las peticiones, enviarlas al servidor y mostrar los datos al cliente. En definitiva, se comporta como la interfaz ([front-end](#) ) que utiliza el usuario para interactuar con el servidor. Las funciones que lleva a cabo el proceso cliente se resumen en los siguientes puntos:
 - Interactuar con el usuario.
 - Procesar las peticiones para ver si son válidas y evitar peticiones maliciosas al servidor.
 - Recibir los resultados del servidor.
 - Formatear y mostrar los resultados.
- **Servidor.** Es el proceso encargado de recibir y procesar las peticiones de los

clientes para permitir el acceso a algún recurso ([back-end](#) ). Las funciones del servidor son:

- Aceptar las peticiones de los clientes.
- Procesar las peticiones.
- Formatear y enviar el resultado a los clientes.
- Procesar la lógica de la aplicación y realizar validaciones de datos.
- Asegurar la consistencia de la información.
- Evitar que las peticiones de los clientes interfieran entre sí.
- Mantener la seguridad del sistema.

La idea es tratar el servidor como una entidad que realiza un determinado conjunto de tareas y que las ofrece como servicio a los clientes.

La forma más habitual de utilizar el modelo cliente/servidor es mediante la utilización de equipos a través de interfaces gráficas; mientras que la administración de datos y su seguridad e integridad se deja a cargo del servidor. Normalmente, el trabajo pesado lo realiza el servidor y los procesos clientes sólo se encargan de interactuar con el usuario. En otras palabras, el modelo Cliente/Servidor es una extensión de programación modular en la que se divide la funcionalidad del software en dos módulos con el fin de hacer más fácil el desarrollo y mejorar su mantenimiento



Autoevaluación

Indica qué operación no realiza un servidor:

- ☐ Recibir la petición del cliente.
- ☐ Procesar la petición del cliente.
- ☐ Mostrar el resultado al usuario.
- ☐ Asegurar el sistema.

Mostrar retroalimentación

Solución

1. Incorrecto (#answer-4_58)
2. Incorrecto (#answer-4_78)
3. Incorrecto (#answer-4_80)
4. Incorrecto (#answer-4_82)

1.1.- Características básicas

Las características básicas de una arquitectura Cliente/Servidor son:

- Combinación de un cliente que interactúa con el usuario, y un servidor que interactúa con los recursos compartidos. El proceso del cliente proporciona la interfaz de usuario y el proceso del servidor permite el acceso al recurso compartido.
- Las tareas del cliente y del servidor tienen diferentes requerimientos en cuanto al procesamiento; todo el trabajo de procesamiento lo realiza el servidor y mientras que el cliente interactúa con el usuario.
- Se establece una relación entre distintos procesos, las cuales se pueden ejecutar en uno o varios equipos distribuidos a lo largo de la red.
- Existe una clara distinción de funciones basada en el concepto de "servicio", que se establece entre clientes y servidores.
- La relación establecida puede ser de muchos a uno, en la que un servidor puede dar servicio a muchos clientes, regulando el acceso a los recursos compartidos.
- Los clientes corresponden a procesos activos ya que realizan las peticiones de servicios a los servidores. Estos últimos tienen un carácter pasivo ya que esperan las peticiones de los clientes.
- Las comunicaciones se realizan estrictamente a través del intercambio de mensajes.
- Los clientes pueden utilizar sistemas heterogéneos ya que permite conectar clientes y servidores independientemente de sus plataformas.



Q WOCinTech Chat (CC BY-SA)



Autoevaluación

¿Qué proceso necesita más recursos?

☐ Verdadero ☐ Falso

Falso

Las respuesta Servidor es la correcta ya que el servidor debe atender y procesar las peticiones de los clientes.

1.2.- Ventajas y desventajas

¿Te has preguntado alguna vez qué ventajas y desventajas el esquema Cliente/Servidor? ¡Veámoslas!

Entre las principales ventajas del esquema Cliente/Servidor destacan:

- Utilización de clientes ligeros (con pocos requisitos hardware) ya que el servidor es quien realmente realiza todo el procesamiento de la información.
- Facilita la integración entre sistemas diferentes y comparte información permitiendo interfaces amigables al usuario.
- Se favorece la utilización de interfaces gráficas interactivas para los clientes para interactuar con el servidor. El uso de interfaces gráficas en el modelo Cliente/Servidor presenta la ventaja, con respecto a un sistema centralizado, de que normalmente sólo transmite los datos por lo que se aprovecha mejor el ancho de banda de la red.
- El mantenimiento y desarrollo de aplicaciones resulta rápido utilizando las herramientas existentes.
- La estructura inherentemente modular facilita además la integración de nuevas tecnologías y el crecimiento de la infraestructura computacional, favoreciendo así la escalabilidad de las soluciones.
- Contribuye a proporcionar a los diferentes departamentos de una organización, soluciones locales, pero permitiendo la integración de la información relevante a nivel global.
- El acceso a los recursos se encuentra centralizado.
- Los clientes acceden de forma simultánea a los datos compartiendo información entre sí.

Entre las principales desventajas del esquema Cliente/Servidor destacan:

- El mantenimiento de los sistemas es más difícil pues implica la interacción de diferentes partes de hardware y de software lo cual dificulta el diagnóstico de errores.
- Hay que tener estrategias para el manejo de errores del sistema.
- Es importante mantener la seguridad del sistema.
- Hay que garantizar la consistencia de la información. Como es posible que varios clientes operen con los mismos datos de forma simultánea, es necesario utilizar mecanismos de sincronización para evitar que un cliente modifique datos sin que lo sepan los demás clientes.



Autoevaluación

¿Qué proceso es el encargado de poder atender las peticiones de varios usuarios de forma concurrente?

- ☐ El cliente.
- ☐ El servidor.
- ☐ El cliente y el servidor.

Mostrar retroalimentación

Solución

1. Incorrecto (#answer-8_58)
2. Incorrecto (#answer-8_84)
3. Correcto (#answer-8_86)

1.3.- Modelos

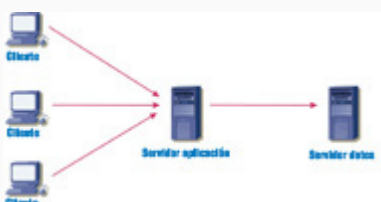
La principal forma de clasificar los modelos Cliente/Servidor es a partir del número de capas (tiers) que tiene la infraestructura del sistema. De ésta forma podemos tener los siguientes modelos:

- **1 capa (1-tier).** El proceso cliente/servidor se encuentra en el mismo equipo y realmente no se considera un modelo cliente/servidor ya que no se realizan comunicaciones por la red.
- **2 capas (2-tiers).** Es el modelo tradicional en el que existe un servidor y unos clientes bien diferenciados. El principal problema de éste modelo es que no permite escalabilidad del sistema y puede sobrecargarse con un número alto de peticiones por parte de los clientes.



Q Ministerio de Educación (Uso educativo nc)

- **3 capas (3-tiers).** Para mejorar el rendimiento del sistema en el modelo de dos capas se añade una nueva capa de servidores. En este caso se dispone de:
 - **Servidor de aplicación.** Es el encargado de interactuar con los diferentes clientes y enviar las peticiones de procesamiento al servidor de datos.
 - **Servidor de datos.** Recibe las peticiones del servidor de aplicación, las procesa y le devuelve su resultado al servidor de aplicación para que éste los envíe al cliente. Para mejorar el rendimiento del sistema, es posible añadir los servidores de datos que sean necesarios.



Q Ministerio de Educación (Uso educativo nc)

- **n capas (n-tiers).** A partir del modelo anterior, se pueden añadir capas adicionales de servidores con el objetivo de separar la funcionalidad de cada servidor y de mejorar el rendimiento del sistema. Éste es el utilizado en los casos de aplicaciones comerciales a gran escala (por ejemplo en sistemas de mensajería en tiempo real, redes sociales y servicios bancarios entre otros).





¿Es realmente el modelo de 1-capa (1-tiers) un modelo cliente/servidor?

☐ Verdadero ☐ Falso

Falso

Realmente no es un modelo cliente/servidor porque los dos procesos están en el mismo equipo y no existen comunicaciones externas.

1.4.- Programación

De forma interna, los pasos que realiza el servidor para realizar una comunicación son:

- **Publicar puerto.** Publica el puerto por donde se van a recibir las conexiones.
- **Esperar peticiones.** En este momento el servidor queda a la espera a que se conecte un cliente. Una vez que se conecte un cliente se crea el socket del cliente por donde se envían y reciben los datos.
- **Envío y recepción de datos.** Para poder recibir/enviar datos es necesario crear un flujo (stream) de entrada y otro de salida. Cuando el servidor recibe una petición, éste la procesa y le envía el resultado al cliente.
- Una vez finalizada la comunicación **se cierra el socket del cliente.**



Los pasos que realiza el cliente para realizar una comunicación son:

- **Conectarse con el servidor.** El cliente se conecta con un determinado servidor a un puerto específico. Una vez realizada la conexión se crea el socket por donde se realizará la comunicación.
- **Envío y recepción de datos.** Para poder recibir/enviar datos es necesario crear un flujo (stream) de entrada y otro de salida.
- Una vez finalizada la comunicación se **cierra el socket.**



Autoevaluación

¿Qué función es la que permite espera la conexión de un cliente?

- ☐ ServerSocket.
- ☐ Socket.
- ☐ Accept.
- ☐ Stream.

Mostrar retroalimentación

Solución

1. Incorrecto (#answer-12_58)
2. Incorrecto (#answer-12_94)
3. Correcto (#answer-12_96)

4. Incorrecto (#answer-12_98)

1.5.- Ejemplo

A modo de ejemplo, que ilustre los apartados anteriores, y retomando el ejemplo del apartado 2.4 *Ejemplo* de la unidad de trabajo anterior, vamos a volver a analizar dicho material donde se utilizan *sockets* para enviar información. Este ejemplo es muy sencillo y trata sobre un proceso servidor que acepta tres clientes (de forma secuencial no concurrente) e indica a cada uno su número de cliente como respuesta.

Implementación del servidor

```
...
ServerSocket socketServidor = new ServerSocket(puerto); // se inicia
                        // la escucha del servidor en el puerto indicado

// mensaje por salida estándar que informa el puerto en el que se
// realiza la escucha
System.out.println("Inicio de escucha en el puerto " + puerto);
for (int numeroClientes = 1; numeroClientes <= 3; numeroClientes++) {
    // espera activa hasta que se conecte un cliente. En ese
    // momento se realiza la creación de un socket para dicho
    // cliente. Hasta que no se conecte un cliente el puerto
    // permanece ocupado y el proceso permanece en estado de
    try (Socket socketCliente = socketServidor.accept()) { // ejecución
        System.out.println("Se está atendiendo al cliente '" + numeroClientes);
        DataOutputStream flujo_salida= new DataOutputStream(socketCliente.getOutputStream());
        flujo_salida.writeUTF("Hola cliente '" + numeroClientes + "'.");
        socketCliente.close(); // cuando se ha finalizado de atender al
                                // cliente se procede a cerrar el socket y
                                // liberar los recursos asociados a éste
    } catch (Exception e){
        System.out.println(e.getMessage());
    }
}
System.out.println("Ya se han atendido a los 3 clientes en el puerto '" + puerto);
...
```

Implementación del cliente

```
...
try {
    Socket socketCliente = new Socket(host, puerto); // primitiva que
        // permite al cliente la conexión, en el host y puerto indicados,
        // con el servidor. Si no hubiera ningún proceso atendiendo
```

```

        // en esa máquina/puerto la petición se dispara una
        // excepción (indicada más abajo).

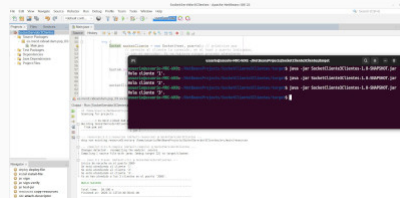
        System.out.println(           // salida por consola del mensaje
            (new DataInputStream(      // enviado desde el servidor
                socketCliente.getInputStream()).readUTF() );

        socketCliente.close(); // cuando se ha finalizado de realizar la
                               // solicitud del cliente se procede a cerrar
                               // el socket y liberar los recursos asociado
                               // a éste
    } catch (IOException e) {
        System.out.println(e.getMessage()); // se imprime por la salida
                                           // estándar el mensaje de error
    }
    ...

```

Hay varias formas de realizar la prueba:

- En una terminal de comandos se ejecuta el servidor y, en otra terminal, se ejecuta 3 veces el cliente



Q Ministerio de Educación (Uso educativo nc)

- En el entorno de desarrollo, en este caso NetBeans, se ejecuta el servidor y se observa el resultado en la pestaña **Output** mientras que, en una terminal de comandos se ejecuta 3 veces la versión del cliente.

En ambos casos, para ejecutar en la consola de comandos, basta con invocar el archivo **.jar** de la siguiente manera: **java -jar Archivo.jar**

En la captura de pantalla se muestra, en la imagen de fondo que ilustra el resultado de ejecución en NetBeans, cómo el servidor ha procesado las solicitudes de tres clientes (la primera) y, en primer plano, la respuesta que obtiene cada uno de éstos tras haberse conectado al servidor.



Código fuente completo del ejemplo

Aquí puedes descargar el código fuente completo:

 [Creación de Servidor que atiende 3 clientes empleando sockets](#) 
(PSP04_CONT_R007_ServidorSecNoConc.zip)

 [Creación de Cliente que recibe saludo desde servidor empleando sockets](#)
 (PSP04_CONT_R008_SocketCliente3Clientes.zip)

2.- Optimización de sockets



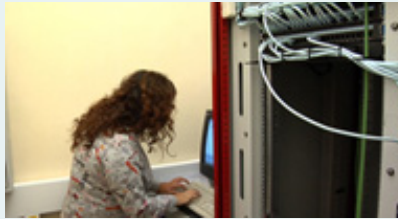
Caso práctico

María y Juan están revisando la aplicación que están desarrollando.

María: Juan, la aplicación va muy bien pero ahora vamos a optimizar su comportamiento. Lo primero que vamos a hacer es hacer que la aplicación permita atender múltiples peticiones de forma concurrente y para ello vamos a utilizar las hebras de ejecución (**threads**).

Antonio: Yo he utilizado antes las hebras cuando quería que un programa hiciera varias cosas a la vez. Pero no sé muy bien como utilizarlas con **sockets**.

María: Es muy fácil. Vamos a hacer que cada vez que se recibe un cliente se cree una hebra para atender dicho cliente. Vamos a verlo.



Q Ministerio de Educación. (Uso educativo [nc](#))

A la hora de utilizar los sockets es muy importante optimizar su funcionamiento y garantizar la seguridad del sistema. Como la información reside en el servidor y existen múltiples clientes que realizan peticiones es totalmente indispensable permitir que la aplicación cliente/servidor cuente con las siguientes características que veremos más adelante:



Q Taavi Randmaa (CC BY-SA)

- **Atender múltiples peticiones simultáneamente.** El servidor debe permitir el acceso de forma simultánea al servidor para acceder a los recursos o servicios que éste ofrece.
- **Seguridad.** Para asegurar el sistema, como mínimo, el servidor debe ser capaz de evitar la pérdida de información, filtrar las peticiones de los clientes para asegurar que éstas están bien formadas y llevar un control sobre las diferentes transacciones de los clientes.
- Por último, es necesario dotar a nuestro sistema de mecanismos para **monitorizar los tiempos de respuesta** de los clientes para ver el comportamiento del sistema.

2.1.- Atender múltiples peticiones simultáneas

Si observamos la siguiente figura, cuando un servidor recibe la conexión del cliente (`accept`) se crea el socket del cliente, se realiza el envío y recepción de datos y se cierra el socket del cliente finalizando la ejecución del servidor.



Como el objetivo es permitir que múltiples clientes utilicen el servidor

Q Ministerio de Educación (Uso educativo nc)

de forma simultánea es necesario que la parte que atiende al cliente (zona coloreada de azul) se atienda de forma independiente para cada uno de los clientes.

Para ello, en vez de ejecutar todo el código del servidor de forma secuencial, vamos a tener un bucle `while` para que cada vez que se realice la conexión de un cliente se cree una hebra de ejecución (thread) que será la encargada de atender al cliente. De ésta forma, tendremos tantas hebras de ejecución como clientes se conecten a nuestro servidor de forma simultánea.

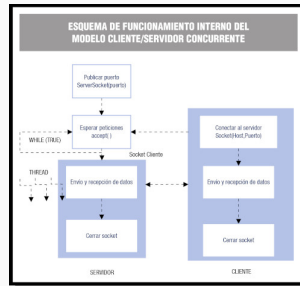
De forma resumida, el código necesario es:

```
while(true){
    socketCliente = socketServidor.accept(); // espera activa hasta
    // que se conecte un cliente. En ese momento se realiza la
    // creación de un socket para dicho cliente
    // Hasta que no se conecte un cliente el puerto permanece
    // ocupado y el proceso permanece en estado de ejecución

    // mensaje por salida estándar que informa en
    System.out.println("Cliente conectado en el puerto " + puerto);
    // el lado del servidor que se ha conectado
    // un cliente

    // Se atiende al cliente mediante un 'thread'
    // Al invocar 'start', dentro del método 'run',
    // se establece el código (lógica de negocio)
    new SocketServidor(socketCliente).start(); // que es necesario
    // para atender la petición recibida por el cliente
}
```

Y a continuación puede ver su representación de forma gráfica:



Q Ministerio de Educación (Uso educativo nc)



Autoevaluación

¿Qué operaciones realiza el thread?

- ☐ ServerSocket, Socket y accept.
- ☐ ServerSocket y accept.
- ☐ Accept y procesado de la información.
- ☐ Procesado de la información.

Mostrar retroalimentación

Solución

1. Incorrecto (#answer-18_58)
2. Incorrecto (#answer-18_111)
3. Incorrecto (#answer-18_113)
4. Correcto (#answer-18_115)



Para saber más

Si deseas más información sobre el uso de *threads* y *sockets* en java puedes consultar la siguiente presentación:

[https://www.youtube.com/
embed/UKcRgwWXQZU](https://www.youtube.com/embed/UKcRgwWXQZU)
([https://www.youtube.com/
embed/UKcRgwWXQZU](https://www.youtube.com/embed/UKcRgwWXQZU))

🔍 Youtube (

Licencia estándar de YouTube)

📄 Descripción textual del vídeo



2.2.- Threads

Para crear un hilo hay que indicar explícitamente que la clase extienda a `Thread`:

```
public class SocketServidor extends Thread{

    public SocketServidor() {
        // Inicialización del hilo
    }

    public static void main( String[] arg ) {
        new SocketServidor().start();
    }

    public void run(){
        // Tareas que realiza el hilo
    }

}
```

donde:

- El método constructor `public SocketServidor` permite inicializar los valores iniciales que recibe el hilo.
- El método `run()` es el encargado de realizar las tareas del hilo.
- Para iniciar el hilo se crea el objeto `SocketServidor` y se inicia:

```
new SocketServidor().start();
```



Para saber más

Si deseas más información sobre la [clase Thread](#) puedes consultar la documentación oficial de Java.

2.3.- Ejemplo I

Si añade al código anterior la utilización de sockets por parte del servidor, tal y como se ha visto anteriormente, el código resultante es un servidor que admite múltiples peticiones concurrentes.

Lógicamente el funcionamiento del cliente no cambia ya que la concurrencia la realiza el servidor.

Implementación del servidor

```
...
try {
    ServerSocket socketServidor = new ServerSocket(puerto); // se ini
        // la escucha del servidor en el puerto indic

        // mensaje por salida estándar que informa
    System.out.println("Inicio de escucha en el puerto " // puerto er
        + puerto);          // que se realiza la escu

    while(true){
        socketCliente = socketServidor.accept(); // espera activa has
            // que se conecte un cliente. En ese momento se realiza l
            // creación de un socket para dicho cliente
            // Hasta que no se conecte un cliente el puerto permanece
            // ocupado y el proceso permanece en estado de ejecución

            // mensaje por salida estándar que informa
        System.out.println("Cliente conectado en el puerto " + puerto
            // el lado del servidor que se ha conectac
            // un cliente

            // Se atiende al cliente mediante un 'thread'
            // Al invocar 'start', dentro del método 'run'
            // se establece el código (lógica de negocio)
        new SocketServidor(socketCliente).start(); // que es necesari
            // para atender la petición recibida por el cliente
    }
} catch (IOException e) {
    System.out.println(e.getMessage()); // se imprime por la salida
        // estándar el mensaje de error
}

/**
 * Ejecución del hilo que atiende a cada cliente
 */
@Override
```

```

public void run(){

    try{

        DataInputStream flujoEntrada = // declaración del flujo de entrada
            new DataInputStream(socketCliente.getInputStream());
        DataOutputStream flujoSalida // declaración del flujo de salida
            = new DataOutputStream(socketCliente.getOutputStream());

        // en este caso de ejemplo enviamos un mensaje al
        // al cliente y no procesamos cualquier posible
        // envío de información que éste haya podido real
        flujoSalida.writeUTF("Cliente: usted se ha conectado de forma cor

                                // cuando se ha finalizado de atender a
        socketCliente.close(); // cliente se procede a cerrar el socket
                                // liberar los recursos asociados a éste

        System.out.println("Cliente desconectado");// se imprime por la salida
                                // estándar el fin de comunicación
    } catch(IOException e ) {
        System.out.println( e.getMessage() );
    }
}

```

Implementación del cliente

```

...
try {
    Socket socketCliente = new Socket(host, puerto); // primitiva que
        // permite al cliente la conexión, en el host y puerto indicados,
        // con el servidor. Si no hubiera ningún proceso atendiendo
        // en esa máquina/puerto la petición se dispara una
        // excepción (indicada más abajo).

    System.out.println(           // salida por consola del mensaje
        (new DataInputStream( // enviado desde el servidor
            socketCliente.getInputStream())).readUTF());

    socketCliente.close(); // cuando se ha finalizado de realizar la
        // solicitud del cliente se procede a cerrar
        // el socket y liberar los recursos asociados
        // a éste
} catch (IOException e) {
    System.out.println(e.getMessage()); // se imprime por la salida
        // estándar el mensaje de error
}

```

```
}  
...
```



Autoevaluación

En la clase `Servidor`, ¿dónde se ubica el código que atiende al cliente?

- ☐ `public Servidor.`
- ☐ `run().`
- ☐ `public static void main.`

Mostrar retroalimentación

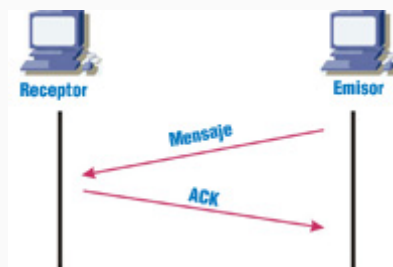
Solución

1. Incorrecto (#answer-24_58)
2. Correcto (#answer-24_119)
3. Incorrecto (#answer-24_121)

2.4.- Pérdida de información

La pérdida de paquetes en las comunicaciones de red es un factor muy importante que hay que tener en cuenta ya que, por ejemplo, si se envía un fichero la pérdida de un único paquete produce que el fichero no se reciba correctamente.

Para evitar la pérdida de paquetes en las comunicaciones, cada vez que se envía un paquete el receptor envía al emisor un paquete de confirmación ACK (*acknowledgement*).

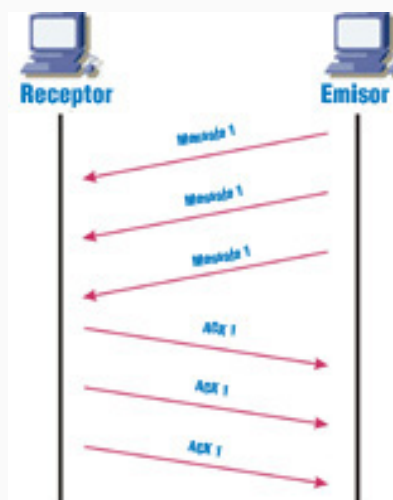


Q Ministerio de Educación (Uso educativo nc)

En caso de que el mensaje no llegue correctamente al receptor el paquete de confirmación no se envía nunca. El emisor, cuando transcurre un determinado tiempo de espera, denominado *timeout*, considera que se ha producido un error en la transmisión del paquete y vuelve a enviarlo.

Este método, aunque efectivo, es poco eficiente ya que para enviar un nuevo paquete debe esperar el ACK del anterior y ello genera un retardo en las comunicaciones.

Una mejora sustancial respecto al método anterior consiste en permitir que el emisor envíe múltiples paquetes sin necesidad de esperar las confirmaciones de cada uno de ellos. De esta forma el emisor puede enviar una ráfaga de N paquetes, mejorando la eficiencia del canal de comunicación.



Q Ministerio de Educación (Uso educativo nc)

Una de las características de las redes de comunicaciones es la posibilidad de que tanto los paquetes, como los ACK, no lleguen en orden o simplemente que se produzca la pérdida de alguno en el transcurso de la transmisión. Por ello es

necesario llevar un control sobre los paquetes enviados y los confirmados mediante recepción de ACK's.

Para llevar un control de los paquetes enviados se utiliza un vector en el que se indica si un determinado paquete se ha enviado correctamente o no. Debido a que los paquetes pueden llegar de forma desordenada, o incluso perderse, es posible encontrar en el vector de configuración múltiples situaciones como la mostrada a continuación:

Vector de ACK (estado inicial)

Paquete	0	1	2	3	4	5	6	7	8	9
ACK	1	1	0	0	1	1	0	0	0	0

En el ejemplo anterior, los paquetes 0, 1, 4, y 5 han llegado correctamente puesto que han sido recibidas las confirmaciones asociadas a éstos. Para continuar con el envío de más paquetes se desplaza el vector de derecha a izquierda con los paquetes enviados correctamente hasta llegar al primero no enviado correctamente (en el caso del ejemplo anterior se trata del paquete numerado con el valor 2). Entonces, siguiendo el ejemplo anterior, el vector es modificado de la siguiente forma:

Vector de ACK (desplazado)

Paquete	2	3	4	5	6	7	8	9	10	11
ACK	0	0	1	1	0	0	0	0	0	0

Al haber desaparecido de la ventana de control los dos primeros elementos, paquetes 0 y 1, el sistema puede enviar más paquetes, en este caso el 10 y el 11, mientras que espera la confirmación de los anteriores.

Es posible observar que el tamaño del vector influye de forma directa en el rendimiento del sistema ya que, cuanto mayor sea su capacidad, más paquetes es posible enviar de forma concurrente. Es necesario señalar que existen limitaciones respecto de esta características como la potencia computacional y, en especial, la cantidad de memoria RAM que es necesario reservar para el vector que implementa la ventana de control.



Para saber más

Si deseas más información sobre los mensajes de confirmación ACK (*acknowledgement*) puedes consultar el siguiente artículo:

2.5.- Transacciones

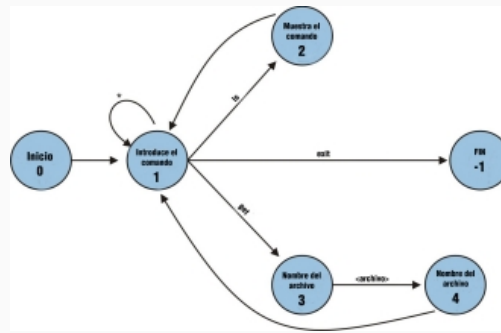
Uno de los principales fallos de seguridad que se producen en la programas cliente/servidor es que el cliente pueda realizar:

- **Operaciones no autorizadas.** El servidor no puede procesar una orden a la que el cliente no tiene acceso. Por ejemplo cuando el cliente realice una solicitud de información a la que no tiene acceso.
- **Mensajes mal formados.** Es posible que el cliente envíe al servidor mensajes mal formados o con información incompleta que produzcan un error de procesamiento del sistema llegando incluso a dejar "bloqueado" el servidor.

Para evitar cualquier problema de seguridad es muy importante modelar el flujo de información y el comportamiento del servidor con un diagrama de estados o autómatas. Por ejemplo, en la siguiente figura, se puede ver que el servidor se inicia en el estado 0 y directamente envía al cliente el mensaje *Introduce el comando*. El cliente puede enviar los comandos:

- *ls* que va al estado 2 mostrando el contenido del directorio y vuelve automáticamente al estado 1.
- *get* que le lleva al estado 3 donde le solicita al cliente el nombre del archivo a mostrar. Al introducir el nombre del archivo se desplaza al estado 4 donde muestra el contenido del archivo y vuelve automáticamente al estado 1.
- *exit* que le lleva directamente al estado donde finaliza la conexión del cliente (estado -1).
- Cualquier otro comando hace que vuelva al estado 1 solicitándole al cliente que introduzca un comando válido.

El servidor, para poder seguir el comportamiento del autómatas, tiene que definir las variables *estado* y *comando*. La variable *estado* almacena la posición en la que se encuentra y la variable *comando* es el comando que recibe el servidor y el que permite la transición de un estado a otro. Cuando se utilizan autómatas muy sencillos, como es el caso del ejemplo, es posible modelar el comportamiento del autómatas utilizando estructuras `case` e `if`. En el caso de utilizar autómatas grandes la mejor forma de modelar su comportamiento es mediante una tabla cuyas filas son los diferentes estados del autómatas y las columnas representan las diferentes entradas del sistema.



Q Ministerio de Educación (Uso educativo nc)



Para saber más

Si deseas más información sobre los autómatas finitos puedes consultar el siguiente enlace:

[Autómata Finito.](#) 



Autoevaluación

¿Qué permite el diagrama de transacciones del sistema?

- ☐ Asegurar el sistema.
- ☐ Implementar los diferentes estados en los que se encuentra el servidor.
- ☐ Evitar que un cliente pueda realizar una operación no autorizada.
- ☐ Todas son correctas.

Mostrar retroalimentación

Solución

1. Incorrecto (#answer-30_58)
2. Incorrecto (#answer-30_129)
3. Incorrecto (#answer-30_131)
4. Correcto (#answer-30_133)

2.6.- Ejemplo II

A continuación, a modo de ejemplo, se muestra la estructura general para implementar el diagrama de transiciones del ejemplo anterior.



Q Ministerio de Educación (Uso educativo nc)

Código fuente del ejemplo:

```
int estado = 1;

do{
    switch(estado){
        case 1:
            flujo_salida.writeUTF("Introduce comando (ls/get/exit)");
            comando = flujo_entrada.readUTF();
            if(comando.equals("ls")){
                System.out.println("\tEl cliente quiere ver el contenido del directorio
                // Mostrar el directorio
                estado = 1;
                break;
            }else{
                if(comando.equals("get")){
                    // Cambiar a estado 3 para mostrar el archivo
                    estado = 3;
                    break;
                }else{
                    estado = 1;
                    break;
                }
            }
        case 3: // mostrar el archivo
            flujo_salida.writeUTF("Introduzca el nombre del archivo:");
            String fichero = flujo_entrada.readUTF();
            // mostrar el archivo
            estado = 1;
            break;
    }
    if(comando.equals("exit"))
        estado = -1;
}while(estado != -1);
```

2.7.- Monitorizar tiempos de respuesta

Un aspecto muy importante para ver el comportamiento de nuestra aplicación Cliente/Servidor son los tiempos de respuesta del servidor. Desde que el cliente realiza una petición hasta que recibe su resultado intervienen dos tiempos:

- **Tiempo de procesamiento:** tiempo que el servidor necesita para procesar la petición del cliente y enviar los datos.
- **Tiempo de transmisión:** tiempo que transcurre para que los mensajes viajen a través de los diferentes dispositivos de la red hasta llegar a su destino.



Q

johnny_automatic (
CC BY-SA)

Para determinar el tiempo de procesamiento tan sólo se necesita medir el lapso que transcurre mientras que el servidor procesa la solicitud del cliente. La lógica que arroja el cálculo anterior, en milisegundos, es posible codificarla empleando el siguiente código:

```
import java.util.Date;
...
long tiempo1 = (new Date()).getTime();
// Procesamiento de la petición del cliente
long tiempo2 = (new Date()).getTime();

System.out.println("\t Tiempo: " + (tiempo2-tiempo1) + " ms");
...
```

Para determinar el tiempo de transmisión es necesario remitir, a través de un mensaje, el tiempo del sistema y, en el receptor (el cliente), se realiza la comparación de su tiempo de respuesta con el que hay contenido en el mensaje. Lógicamente para poder comparar los tiempos de respuesta de dos equipos es totalmente necesario que los relojes del sistema estén sincronizados a través de cualquier servicio de tiempo (NTP). En equipos Windows la sincronización de los relojes se realiza automáticamente y en equipos GNU/Linux se realiza ejecutando el siguiente comando (habiendo instalado previamente el servicio con el comando `sudo apt install ntpdate`):

```
sudo /usr/sbin/ntpdate -u 0.centos.pool.ntp.org
```



Para saber más

Para sincronizar el reloj de dos equipos es necesario utilizar el servicio NTP (Network Time Protocol):

Servicio NTP.

Otra forma de calcular el tiempo de transmisión es utilizar el comando ping.

Como se puede ver en la siguiente captura de pantalla el tiempo para que el cliente origen acceda al servidor www.google.es es de unos 9,55 milisegundos de media.

```
root@kali:~/# ping google.es
ping google.es: 64 bytes from 142.250.150.100: icmp_seq=1 ttl=64 time=9.55 ms
64 bytes from 142.250.150.100: icmp_seq=2 ttl=64 time=9.46 ms
64 bytes from 142.250.150.100: icmp_seq=3 ttl=64 time=9.7 ms
64 bytes from 142.250.150.100: icmp_seq=4 ttl=64 time=9.57 ms
64 bytes from 142.250.150.100: icmp_seq=5 ttl=64 time=9.42 ms
64 bytes from 142.250.150.100: icmp_seq=6 ttl=64 time=9.52 ms
^C
--- google.es ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 588ms
rtt min/avg/max/mdev = 8.872/9.552/13.264/1.723 ms
root@kali:~/#
```

Q Ministerio de Educación. (Uso educativo nc)



Autoevaluación

¿Qué tiempos del sistema afectan en el modelo cliente/servidor?

- ☐ Tiempo de procesamiento.
- ☐ Tiempo de transmisión.
- ☐ Todas son correctas.

Mostrar retroalimentación

Solución

1. Incorrecto (#answer-35_58)
2. Incorrecto (#answer-35_138)
3. Correcto (#answer-35_140)

2.8.- Ejemplo III

A continuación vamos a ver un ejemplo en el que se calcula el tiempo de transmisión de datos entre una aplicación Cliente y Servidor. Para ello, el servidor le va a enviar al cliente un mensaje con el tiempo del sistema en milisegundos y el cliente cuando reciba el mensaje calculará la diferencia entre el tiempo de su sistema y el del mensaje.



Q Ministerio de Educación (Uso educativo nc)

Código fuente del ejemplo:

Implementación del Servidor

```
import java.io.DataOutputStream;
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.Date;

public class Servidor {

    static final int PUERTO = 2000;

    public Servidor() {
        try {
            // inicio de ejecución del servidor en el puerto
            ServerSocket sServidor = new ServerSocket(PUERTO);
            System.out.println("Escucha en el puerto: " + PUERTO);

            // conexión de un cliente
            Socket socketCliente = sServidor.accept();
            System.out.println("Cliente conectado");

            // creación de flujo de salida
            DataOutputStream flujo_salida =
                new DataOutputStream(socketCliente.getOutputStream());

            // cálculo:
            long tiempo1 = (new Date()).getTime();
            flujo_salida.writeUTF(Long.toString(tiempo1));
```

```

        // liberación de recursos (cierre de conexión)
        socketCliente.close();
        System.out.println("Cliente desconectado");
    } catch (IOException e) {
        System.out.println(e.getMessage());
    }
}

public static void main(String[] args) {
    new Servidor();
}
}

```

Implementación del cliente

```

public class Cliente {

    static final String HOST = "localhost";
    static final int PUERTO = 2000;

    public Cliente() {

        String datos;
        Socket socketCliente;

        try {
            // conexión al puerto
            socketCliente = new Socket(HOST, PUERTO);

            // creación de flujo de entrada
            DataInputStream flujo_entrada =
                new DataInputStream(socketCliente.getInputStream());

            // cálculo:
            datos = flujo_entrada.readUTF();
            long tiempo1 = Long.parseLong(datos);
            long tiempo2 = (new Date()).getTime();
            System.out.println("\n El tiempo es:" + (tiempo2 - tiempo1));

            // liberación de recursos (cierre de conexión)
            socketCliente.close();
        } catch (IOException | NumberFormatException e) {
            System.out.println(e.getMessage());
        }
    }
}

```

```
public static void main(String[] args) {  
    new Cliente();  
}  
}
```